

recuperer_sorties

title: “Case 5 — Récupérer une sortie” subtitle: “De la console aux objets R réutilisables”
author: “SeRiouS” format: beamer: aspectratio: 169 theme: Madrid colortheme: seahorse
fontsize: 10pt execute: echo: true eval: false ———

Objectif de la case

Dans les cases précédentes, les résultats statistiques étaient surtout **lus dans la console**.

Dans cette case, l’objectif est de passer à une logique différente :

[sortie affichée objet R réutilisable]

Ce changement est essentiel pour construire ensuite des prompts contrôlés.

Pourquoi récupérer les sorties ?

Une sortie console est utile pour lire un résultat, mais elle est difficile à réutiliser directement.

Un objet R peut être :

- inspecté ;
- découpé ;
- nommé ;
- transformé ;
- inséré dans un prompt ;
- transmis à une autre fonction.

C’est la première étape vers un workflow reproductible.

Les objets de départ

La case utilise deux objets créés précédemment :

```
res_lm_fm  
res_aovsum
```

Ces objets contiennent les résultats produits par :

```
FactoMineR::LinearModel()  
FactoMineR::AovSum()
```

L'objectif est maintenant de regarder ce qu'ils contiennent.

Inspecter un objet R avec `names()`

On commence par afficher les composantes disponibles :

```
names(res_lm_fm)  
names(res_aovsum)
```

La fonction `names()` permet de voir les éléments internes d'un objet R.

Elle répond à la question :

Quelles sorties puis-je récupérer dans cet objet ?

Principe général

Un objet R peut contenir plusieurs sous-objets.

On les récupère avec l'opérateur `$`.

Exemple :

```
lm_ftest <- res_lm_fm$Ftest
```

Cette ligne signifie :

Je prends la partie `Ftest` de l'objet `res_lm_fm` et je la stocke dans un nouvel objet appelé `lm_ftest`.

Sorties récupérées pour `LinearModel()`

À partir de `res_lm_fm`, la case extrait :

```
lm_ftest <- res_lm_fm$Ftest  
lm_ttest <- res_lm_fm$Ttest  
lm_resume <- res_lm_fm$lmResult
```

Ces objets correspondent à trois niveaux de lecture du modèle.

Que contiennent ces objets ?

Objet	Rôle
<code>lm_ftest</code>	Test global des effets du modèle
<code>lm_ttest</code>	Tests associés aux coefficients du modèle
<code>lm_resume</code>	Résumé du modèle linéaire

Le point important est que ces résultats ne sont plus seulement affichés : ils existent maintenant comme objets R.

Sorties récupérées pour `AovSum()`

À partir de `res_aovsum`, la case extrait :

```
aov_ftest <- res_aovsum$Ftest  
aov_ttest <- res_aovsum$Ttest
```

Ces objets permettent de séparer deux niveaux d'analyse.

Deux niveaux d'interprétation

Objet	Question statistique
<code>aov_ftest</code>	Quels effets globaux sont significatifs ?
<code>aov_ttest</code>	Quels coefficients ou contrastes contribuent à ces effets ?

Cette distinction est importante pour éviter de confondre :

- un effet global ;
- un coefficient particulier ;
- une interaction ;
- une modalité ou un contraste.

Pourquoi créer une fonction d’affichage ?

La case définit une petite fonction :

```
afficher_sortie <- function(titre, commande, objet) {
  cat("\n===== \n")
  cat(titre, "\n")
  cat("Objet créé : ", commande, "\n", sep = "")
  cat("===== \n")
  print(objet)
}
```

Cette fonction ne fait pas d’analyse statistique.

Elle sert à rendre la sortie plus lisible.

Rôle pédagogique de afficher_sortie()

La fonction ajoute trois informations autour de chaque objet :

1. un titre explicite ;
2. la commande qui a créé l’objet ;
3. l’objet imprimé.

Cela aide l’étudiant à relier :

[commande R objet créé résultat affiché]

Exemple

```
afficher_sortie(
  titre = "AovSum : effets principaux et interaction - Ftest",
  commande = "aov_ftest <- res_aovsum$Ftest",
  objet = aov_ftest
)
```

Cette instruction rappelle explicitement :

- ce qui est affiché ;
- d'où vient l'objet ;
- à quoi il correspond.

Capturer une sortie complète sous forme de texte

La case crée ensuite deux objets textuels :

```
texte_linearmodel <- paste(  
  capture.output(print(res_lm_fm)),  
  collapse = "\n"  
)  
  
texte_aovsum <- paste(  
  capture.output(print(res_aovsum)),  
  collapse = "\n"  
)
```

Ces objets sont très importants pour la suite du plateau.

Que fait `capture.output()` ?

La fonction `capture.output()` capture ce qui aurait été imprimé dans la console.

Autrement dit, elle transforme une sortie affichée en vecteur de texte.

Exemple :

```
capture.output(print(res_aovsum))
```

permet de récupérer la sortie complète de `res_aovsum` sans se contenter de la lire à l'écran.

Pourquoi utiliser `paste(..., collapse = "\n")` ?

`capture.output()` renvoie souvent plusieurs lignes de texte.

La fonction `paste(..., collapse = "\n")` rassemble ces lignes dans une seule chaîne de caractères, en conservant les retours à la ligne.

On obtient donc un objet texte prêt à être utilisé dans un prompt.

Les objets textuels créés

Objet	Contenu
<code>texte_linearmodel</code>	sortie complète de <code>LinearModel()</code> transformée en texte
<code>texte_aovsum</code>	sortie complète de <code>AovSum()</code> transformée en texte

Ces objets ne sont pas seulement des résultats : ce sont des matériaux pour l'étape suivante.

Une sortie devient un matériau de prompt

Après cette case, on ne dépend plus uniquement de la console.

On dispose de sorties structurées et de textes capturés :

```
lm_ftest  
lm_ttest  
lm_resume  
aov_ftest  
aov_ttest  
texte_linearmodel  
texte_aovsum
```

Ces objets peuvent être injectés dans une consigne adressée à un modèle de langage.

Ce qu'il faut retenir

La case 5 introduit un changement de posture.

Avant :

```
Je lis une sortie dans la console.
```

Après :

```
Je récupère une sortie, je la nomme, je la transforme et je peux la réutiliser.
```

C'est ce qui rend possible un workflow plus contrôlé.

Objets disponibles pour la suite

À la fin de la case, les objets suivants sont disponibles :

Objet	Usage futur
<code>lm_ftest</code>	interpréter les effets globaux du modèle linéaire
<code>lm_ttest</code>	interpréter les coefficients du modèle linéaire
<code>lm_resume</code>	conserver le résumé du modèle
<code>aov_ftest</code>	interpréter les effets principaux et interactions
<code>aov_ttest</code>	interpréter les coefficients et contrastes
<code>texte_linearmodel</code>	alimenter un prompt
<code>texte_aovsum</code>	alimenter un prompt

Transition vers la case suivante

La case suivante utilisera ces objets pour construire un prompt.

L'idée centrale est :

[résultat statistique texte contrôlé prompt interprétable]

On ne demande donc pas au modèle de langage d'inventer une analyse.

On lui fournit un matériau extrait, nommé et contrôlé.

Message clé

Une bonne interprétation assistée par IA ne commence pas par un prompt.

Elle commence par des objets R bien construits.

```
objet statistique
→ sortie récupérée
→ texte capturé
→ prompt contrôlé
→ interprétation inspectable
```